AIORI-3 HACKATHON | SMART CITIES TRACK
Problem Statement PS-003

Team number- A3-PS003-TC019

TRAFFIX

AI-Powered Federated Traffic-Flow Prediction with Communication-Cost Evaluation

Project Documentation

Team Traffix

Mentor: Dr. Shamanth N
Student 1: Anshika Sharon Sonwani
Student 2: Freida B Rodrigues

GitHub Repository
https://github.com/aiori-hackathon/A3-PS003-TC019

Traffix – Towards Smarter Traffic Prediction Through Federated Learning

Table of Contents

1. Introduction and Problem Statement
2. Project Overview and Key Features
3. System Architecture
4. Vehicle Detection and Traffic Data Extraction
5. Data Cleaning and Preprocessing
6. Virtual Junction Partitioning (IID and Non-IID)
7. Chronological Split and Sliding-Window Dataset
8. Learning Approaches and Evaluation Framework
9. Results and Analysis
10. Visualizations
11. Verification and Testing
12. Repository, Reproducibility and Version Control
13. Limitations
14. Future Scope
15. Conclusion

1. Introduction and Problem Statement

1.1 Background

Urban traffic in rapidly growing cities such as Bengaluru is highly dynamic. Vehicle volumes and the mix of vehicle types change from junction to junction and from hour to hour. Accurate short-term traffic-flow prediction helps city authorities plan signal timings, manage congestion and respond to incidents. Conventional machine-learning pipelines for this task collect data from every monitoring location into a single central server. In practice this raises concerns about bandwidth, storage, data ownership and privacy, particularly when the raw data consists of video or images captured by roadside cameras.

Federated Learning (FL) offers a distributed alternative. Each monitoring location trains a model on its own local data and shares only model parameters with a central aggregator, which combines them into a shared global model. The raw data never leaves its source. The trade-off is that FL introduces its own communication cost, because model updates must be exchanged over multiple rounds, and it can struggle when data across locations is heterogeneous (Non-IID).

1.2 Problem Statement (PS-003)

Federated Traffic-Flow Prediction with Communication-Cost Evaluation. The task is to predict urban traffic patterns across multiple junctions while evaluating (a) how well federated learning performs compared with a centralized approach, (b) how performance changes under IID and Non-IID data distributions, and (c) how much communication overhead federated training requires.

1.3 Objectives

- Extract vehicle counts from traffic video data using a computer-vision model.
- Clean, validate and chronologically order the extracted traffic-flow data.
- Distribute the data across four virtual Bengaluru junctions under both IID and Non-IID settings.
- Build a leak-free chronological train/test split and a sliding-window forecasting dataset.
- Establish a centralized learning baseline and compare it with federated learning using Federated Averaging (FedAvg).
- Evaluate prediction quality with MAE and RMSE, and analyse communication cost and convergence.
- Present traffic patterns and model behaviour through interactive visualizations.

2. Project Overview and Key Features

Traffix is an end-to-end traffic-flow prediction platform developed for the AIORI-3 Hackathon (Smart Cities Track). It integrates vehicle detection, traffic-data preprocessing, multi-junction data partitioning, centralized machine learning and federated learning into a single workflow that simulates real-world distributed traffic monitoring. The platform demonstrates how federated learning can support distributed traffic prediction while reducing the need to centralize data from multiple traffic junctions.

2.1 Key Features

Feature	Description
Vehicle detection	Vehicle detection and traffic-data extraction from video datasets using YOLO26n
Data cleaning	Deduplication, path normalization, validation and chronological ordering of traffic records
Junction distribution	Traffic data distributed across four virtual Bengaluru junctions
IID / Non-IID partitioning	Both balanced and heterogeneous client distributions for FL experiments
Chronological split	Strict 80/20 temporal train/test split with no future-data leakage
Centralized baseline	Pooled-data model used as the performance benchmark
Federated learning	Federated Averaging (FedAvg) across the four junction clients
Prediction and evaluation	Next-step traffic-flow prediction evaluated with MAE and RMSE
Cost and convergence analysis	Communication-cost measurement and convergence behaviour across rounds
Visualization	Interactive and static plots of traffic patterns and model performance

Table 1: Key features of the Traffix platform

3. System Architecture

Traffix is organized as a sequential pipeline in which each stage consumes the output of the previous one. The pipeline is implemented in Python and each stage is encapsulated in its own module in the repository.

3.1 Pipeline Stages

1. **Detection and extraction.** YOLO26n detects vehicles in video frames and produces per-frame vehicle counts (outputs/vehicle_counts.csv).
2. **Cleaning and sequencing.** Raw annotations are deduplicated, normalized and ordered by frame ID (src/data_prep.py).
3. **Junction partitioning.** The cleaned data is split across four virtual junctions in IID and Non-IID modes (src/partition.py).
4. **Dataset construction.** A chronological train/test split, leak-free scaling and sliding-window generation are applied (src/dataset.py).
5. **Model training.** A centralized baseline and a FedAvg-based federated model are trained and compared.
6. **Evaluation and visualization.** Charts and reports are generated automatically (src/visualize.py).

3.2 Code Modules

File	Responsibility
src/data_prep.py	Cleaning, deduplication, frame-ID extraction and chronological ordering
src/partition.py	Virtual junction partitioning in IID and Non-IID modes
src/dataset.py	Chronological split, TrafficScaler and sliding-window PyTorch datasets
src/visualize.py	Generation of charts and partition metadata
tests/test_pipeline.py	Unit tests for leakage, scaling, tensor shapes and IID balance

Table 2: Repository modules and their roles

4. Vehicle Detection and Traffic Data Extraction

4.1 Dataset and Model

Vehicle detection uses the IISc BMD-45 dataset (Hugging Face repository iisc-aim/BMD-45), which contains 35,792 traffic images with 373,132 COCO-format bounding-box annotations across 13 vehicle classes. For the detection experiments a 2,000-image subset was selected to keep training manageable on CPU-only Google Colab resources: 1,600 images for training, 200 for validation and 200 for testing. The detector is YOLO26n, a lightweight YOLO model, trained through the Ultralytics framework on PyTorch with an input size of 416 × 416 and a batch size of 2.

Detection outputs (bounding boxes, class labels, confidence scores and per-class counts) are aggregated per frame. These counts form the traffic-flow time series on which the prediction models are subsequently trained.

4.2 Federated Detection Experiment

As a supporting experiment, the detection model itself was trained in a federated fashion. The 1,600 training images were divided into four partitions of 400 images, simulating four clients. Starting from a centralized model trained for three epochs, each client trained for one local epoch and the weights were combined with a single FedAvg round. Both models were evaluated on the same 200-image test set.

Metric	Centralized	Federated
Precision	0.2765	0.2793
Recall	0.3727	0.3677
mAP@50	0.2558	0.2729
mAP@50–95	0.1787	0.1944

Table 3: Detection performance on the 200-image test set

The federated model achieved higher mAP@50 (relative increase of about 6.7%) and mAP@50–95 (about 8.8%), with slightly higher precision and slightly lower recall. These figures describe one particular run and do not prove that federated training will always outperform centralized training.

Communication for this one-round experiment totalled 41.449 MB: client uploads of 20.662 MB and global-model downloads of 20.787 MB, with an individual client model of about 5.166 MB and a global model of about 5.197 MB. This measurement is later used as a reference for the communication-cost model in Section 8.

5. Data Cleaning and Preprocessing

The raw YOLO annotation output (outputs/vehicle_counts.csv) contained 25,551 records. Many frames appeared twice because the same file was referenced under different path prefixes (for example images_000/ versus the root path). The cleaning module normalizes these prefixes and removes the duplicates.

5.1 Cleaning Steps

- **Deduplication:** File-path prefixes were normalized, removing 4,991 duplicate records and leaving 20,560 unique frames.
- **Chronological ordering:** Integer frame IDs (ranging from 41 to 578,344) were extracted so that time steps are ordered monotonically and time flows strictly forward.
- **Output:** Cleaned data is written to outputs/cleaned_traffic_flow.csv (20,560 rows).

5.2 Traffic Composition

Vehicle category	Count	Share
Cars	48,096	51.6%
Two-wheelers / Motorcycles	21,074	22.6%
Trucks / Commercial	15,719	16.9%
Buses	8,353	9.0%
Total vehicles detected	93,242	100%

Table 4: Vehicle composition across the 20,560 cleaned frames

The average traffic flow is 4.54 vehicles per frame (standard deviation 3.36, maximum 24). The fairly wide spread relative to the mean indicates considerable variation between frames, which makes next-step prediction a non-trivial task.

6. Virtual Junction Partitioning (IID and Non-IID)

To simulate distributed traffic monitoring, the cleaned dataset is divided among four virtual junctions representing distinct Bengaluru mobility zones. The junction names are illustrative labels for simulated clients; they are not real connected cameras or location-specific recordings.

6.1 Virtual Junctions

Junction	Name	Simulated profile
Junction 1	Silk Board Arterial	High-volume commuter corridor, car/cab dominant
Junction 2	Majestic Urban Core	Dense city centre, two-wheeler/motorcycle dense
Junction 3	Hebbal Ring Road	Freight and logistics corridor, high truck and bus ratio
Junction 4	Indiranagar Central	Mixed commercial/residential grid, balanced flow

Table 5: Simulated Bengaluru junctions

6.2 IID Partition

In the IID setting the frames are distributed so that every junction receives a statistically similar share of the data. Sample counts are nearly equal and the vehicle mix at each junction closely matches the global composition.

Junction	Name	Samples	Mean flow	Car %	2W %	Bus / Truck %
1	Silk Board Arterial	5,152	4.57	51.8	22.3	9.0 / 16.8
2	Majestic Urban Core	5,136	4.44	51.1	23.2	8.9 / 16.9
3	Hebbal Ring Road	5,136	4.54	51.4	22.9	8.7 / 17.0
4	Indiranagar Central	5,136	4.60	52.1	22.0	9.1 / 16.8

Table 6: IID partition statistics

6.3 Non-IID Partition

In the Non-IID setting each junction is given a distinct profile, so both the amount of data and the vehicle mix differ substantially between clients. This reproduces the heterogeneity expected in real deployments and is the more challenging scenario for FedAvg.

Junction	Name	Samples	Mean flow	Car %	2W %	Bus / Truck %
1	Silk Board Arterial	6,901	5.01	60.2	18.0	7.6 / 14.2
2	Majestic Urban Core	4,351	4.48	43.0	36.1	6.7 / 14.2
3	Hebbal Ring Road	4,847	4.37	45.4	17.8	13.2 / 23.6
4	Indiranagar Central	4,461	4.04	51.6	22.4	9.1 / 16.9

Table 7: Non-IID partition statistics

6.4 IID versus Non-IID Comparison

Aspect	IID	Non-IID
Sample distribution	Nearly equal (5,136–5,152)	Unequal (4,351–6,901)
Car share range	51.1% – 52.1%	43.0% – 60.2%
Two-wheeler share range	22.0% – 23.2%	17.8% – 36.1%
Truck share range	16.8% – 17.0%	14.2% – 23.6%
Mean flow range	4.44 – 4.60	4.04 – 5.01
Expected FL difficulty	Low – clients are similar	High – client drift likely

Table 8: Comparison of the two partitioning strategies

7. Chronological Split and Sliding-Window Dataset

7.1 Chronological Train/Test Split

Traffic flow is a time series, so a random split would allow the model to learn from the future. Traffix instead applies a strict temporal split: the first 80% of the timeline is used for training and the remaining 20% for testing, so that the latest training timestamp is always earlier than the earliest test timestamp (train_max_time < test_min_time). The pooled centralized baseline contains 16,448 training frames and 4,112 test frames, totalling the 20,560 cleaned frames.

7.2 Leak-Free Scaling

A TrafficScaler is fitted only on the training split and then applied unchanged to the test split. This prevents statistics from the test period from influencing the normalization of the training data.

7.3 Sliding-Window Generator

Training samples are produced with a sliding window. The model observes a lookback window of W = 12 consecutive time steps and predicts the traffic flow of the next step (horizon H = 1). With a batch size of 32 the data loader yields the following tensors:

Tensor	Shape	Meaning
Input X	[32, 12, 5]	Batch, window length, features per time step
Target y	[32, 1]	Batch, forecast horizon

Table 9: Data-loader tensor shapes (verified for training and test batches)

7.4 Centralized Baseline Data

For benchmarking, all four junctions are pooled into a single global dataset. The centralized model is trained on this pooled data and represents the best case in which all data is available at one location. Federated models are judged against this baseline.

8. Learning Approaches and Evaluation Framework

8.1 Experimental Configurations

Configuration	Data setting	Training procedure
Centralized baseline	All junction data pooled	Single model trained on the pooled training split
Federated – IID	Four IID junction clients	Local training per junction, FedAvg aggregation
Federated – Non-IID	Four Non-IID junction clients	Local training per junction, FedAvg aggregation

Table 10: Experimental configurations

8.2 Federated Averaging (FedAvg)

In each round the server sends the current global model to all junction clients. Each client trains on its own local training data and returns the updated weights. The server then forms the new global model as a weighted average, where each client is weighted by its number of training samples:

$$W_global = \Sigma\ (n_k / N) \cdot W_k$$

Here W_k is the model of client k, n_k its number of training samples and N the total number of training samples. With IID partitions the weights are almost equal; with Non-IID partitions Junction 1 contributes more than the others because it holds more data.

8.3 Evaluation Metrics

- **Mean Absolute Error (MAE):** The average absolute difference between predicted and actual traffic flow. It is easy to interpret in vehicles per frame.
- **Root Mean Squared Error (RMSE):** The square root of the mean squared error. It penalizes large errors more heavily and highlights poor predictions during traffic peaks.
- **Communication cost:** The amount of data exchanged between clients and server during training.
- **Convergence:** How the global-model error evolves across communication rounds and how many rounds are needed to stabilize.

8.4 Communication-Cost Model

In every round each of the K participating clients uploads its model and downloads the global model. If S is the model size, the total communication after R rounds is:

$$C_total = R \times K \times 2 \times S$$

This formula is consistent with the measured detection experiment in Section 4.2: with K = 4 clients, one round and a model of roughly 5.2 MB, the uploads (4 × 5.166 MB) and downloads (4 × 5.197 MB) sum to about 41.4 MB. Because only model parameters are exchanged, the cost depends on model size and number of rounds rather than on the volume of raw traffic data, which is the central trade-off examined by this project.

9. Results and Analysis

9.1 Completed Pipeline Outputs

Item	Result
Raw records → unique frames	25,551 → 20,560 (4,991 duplicates removed)
Frame ID range	41 to 578,344
Total vehicles detected	93,242
Average flow per frame	4.54 (std 3.36, max 24)
Centralized train / test frames	16,448 / 4,112
Window configuration	W = 12 lookback, H = 1 forecast
Random seed	42
Unit tests	4 tests passed in 0.043 s

Table 11: Summary of validated pipeline outputs

9.2 Prediction Performance and Communication Cost

The table below records the final comparison between the centralized baseline and the federated configurations. Values should be entered from the final training run of the repository.

Configuration	MAE	RMSE	Rounds to converge	Total comm. cost (MB)
Centralized baseline	[ ]	[ ]	N/A	N/A
Federated – IID	[ ]	[ ]	[ ]	[ ]
Federated – Non-IID	[ ]	[ ]	[ ]	[ ]

Table 12: Final prediction and communication results (to be completed from the final run)

9.3 Expected Analysis Points

- Federated models trained on IID partitions are expected to approach the centralized baseline, since each client sees a representative sample of traffic.
- Non-IID partitions are expected to slow convergence and may increase error, because local updates pull the model towards each junction's own vehicle mix.
- The additional communication cost of FL should be weighed against the benefit of never moving raw junction data to a central server.

10. Visualizations

Visualization scripts generate publication-quality charts, saved in outputs/splits/plots/, together with partition metadata in partition_metadata.json.

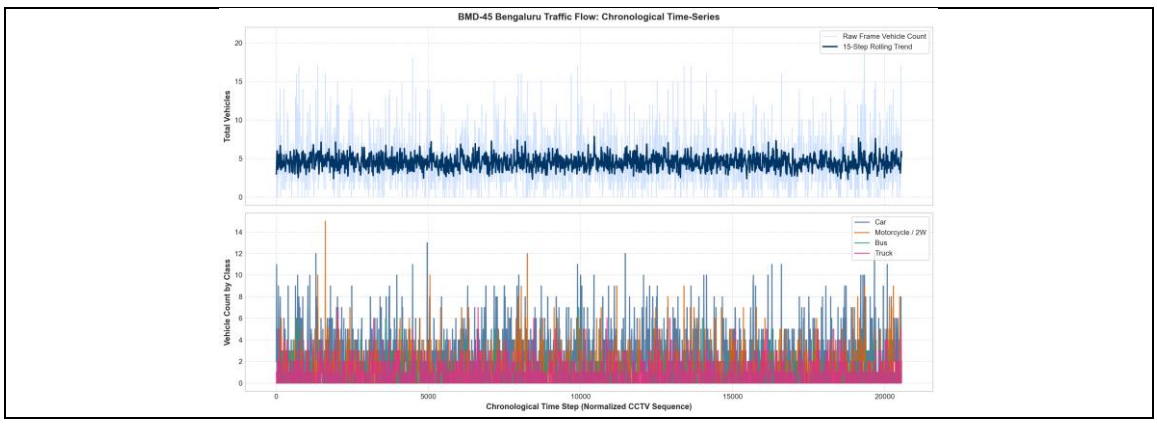


Figure 1: Full traffic time series with a 15-step rolling trend

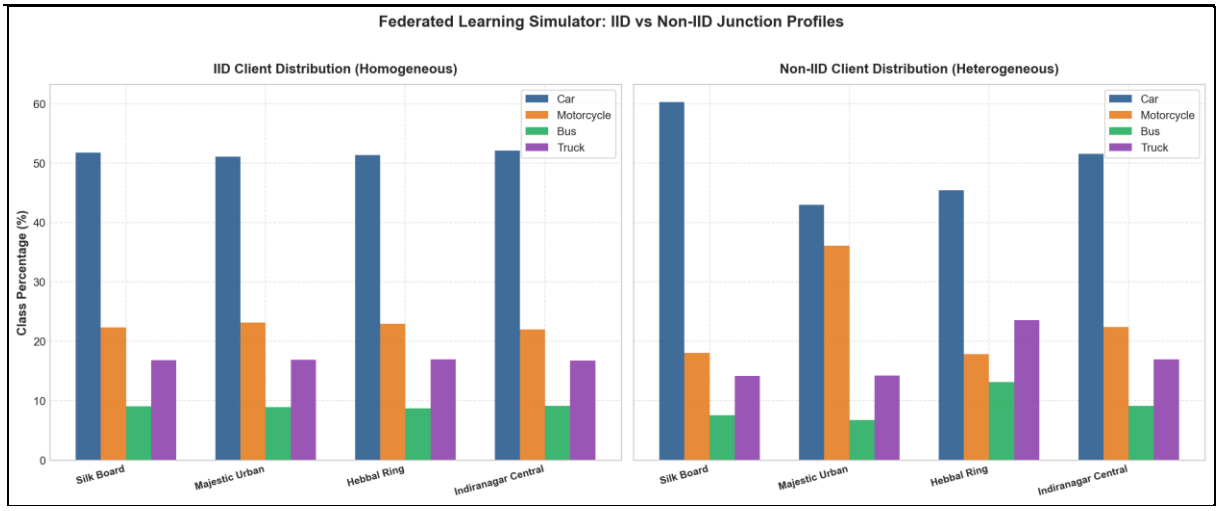


Figure 2: Vehicle-class proportions across junctions, IID vs Non-IID

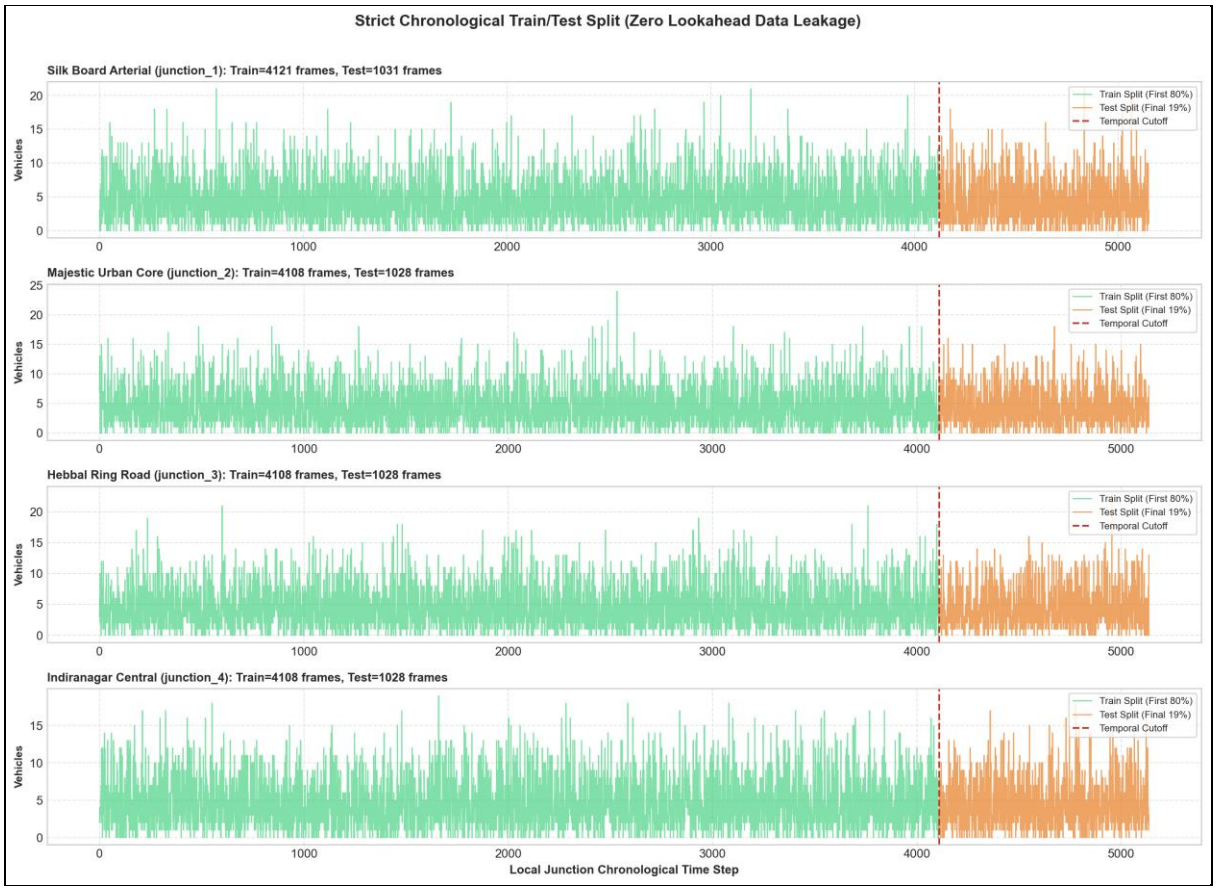


Figure 3: Chronological train/test boundary per junction (IID)

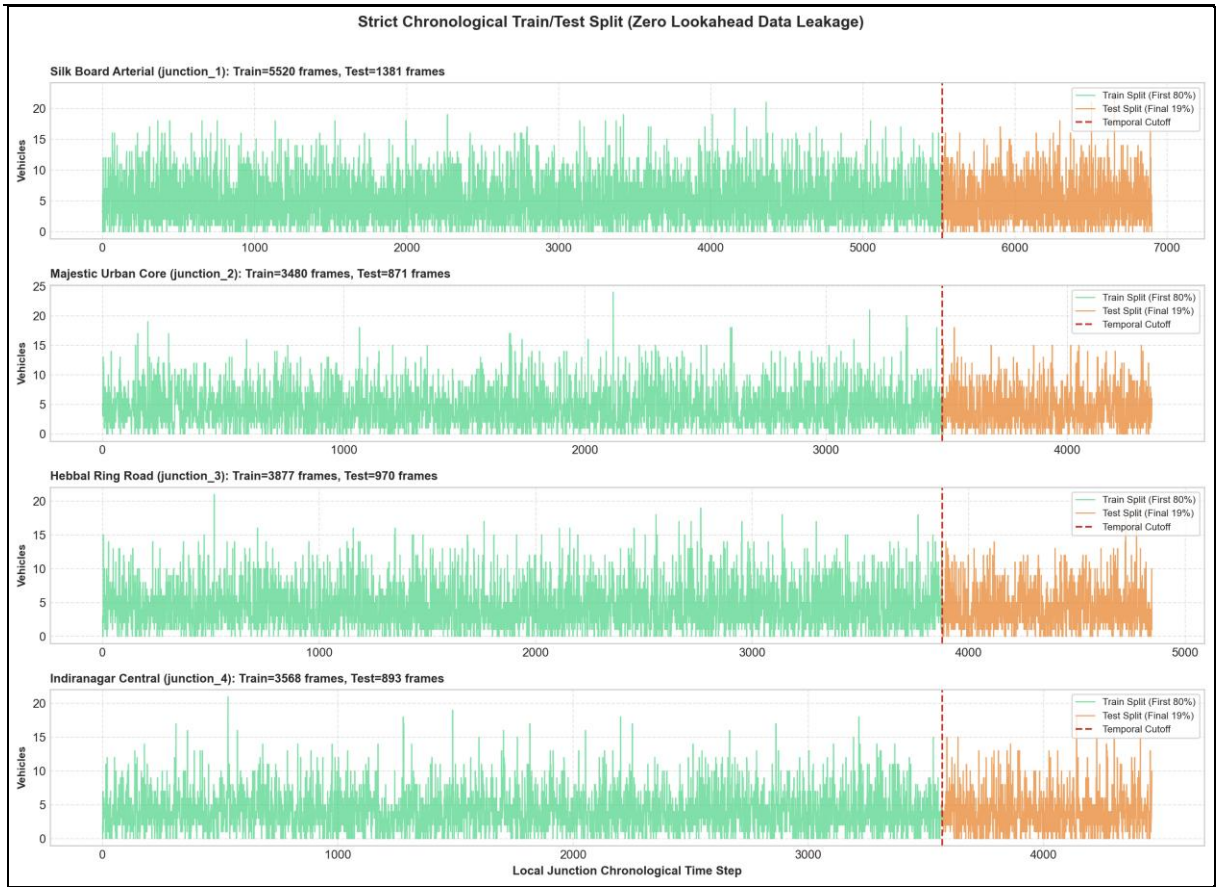


Figure 4: Chronological train/test slices per junction (Non-IID)

11. Verification and Testing

The pipeline is covered by four automated unit tests in tests/test_pipeline.py. All four tests passed in 0.043 seconds.

7. **Zero temporal leakage:** Confirms that the training period ends before the test period begins.
8. **Scaler fitted on training data only:** Confirms that the TrafficScaler statistics come from the training split only.
9. **Tensor dimensions:** Confirms that the sliding-window generator produces the expected (X, y) shapes.
10. **IID balance:** Confirms that IID clients receive balanced data.

12. Repository, Reproducibility and Version Control

- **Repository:** github.com/aiori-hackathon/A3-PS003-TC019
- **Latest commit documented:** 56fdafc – feat(data-prep): implement virtual junction partitioning (IID & Non-IID) and chronological train/test split, pushed to origin/main.
- **Reproducibility:** A fixed random seed of 42 is used throughout the pipeline.
- **Repository hygiene:** .gitignore was updated to prevent accidental commits of heavy annotated frames (outputs/annotated_images/).
- **Technology stack:** Python, PyTorch, Ultralytics YOLO26n, FedAvg, Google Colab, Git and GitHub.

13. Limitations

- Junctions are simulated partitions of one dataset, not independent physical cameras or real network clients.

- The detection experiment used a 2,000-image subset, CPU-only training, one local epoch and a single federated round.
- Vehicle counts depend on detector accuracy; the detection metrics (mAP@50 of about 0.27) indicate that counts contain noise.
- Non-IID profiles are constructed to mimic urban patterns and may not capture all real-world variation.
- The platform is a research prototype and not a production traffic-management system.

14. Future Scope

- **Real-time monitoring:** Integrate live CCTV or video streams for continuous detection and prediction.
- **Edge deployment:** Deploy independent client models on edge devices at physical junctions.
- **Multiple rounds:** Run longer federated training to study convergence in depth.
- **Advanced aggregation:** Study FedProx or personalization to handle strongly Non-IID junctions.
- **Privacy enhancements:** Explore secure aggregation and differential privacy.
- **Richer forecasting:** Extend to longer forecast horizons, congestion indicators and multi-step prediction.

15. Conclusion

Traffix delivers a complete, validated pipeline for federated traffic-flow prediction: vehicle detection and data extraction, cleaning and chronological sequencing, IID and Non-IID distribution across four virtual Bengaluru junctions, a leak-free temporal split and sliding-window dataset, and the groundwork for a rigorous centralized-versus-federated comparison using MAE, RMSE, communication cost and convergence. The supporting detection experiment shows that federated aggregation is practical for the project's models at a modest communication cost of about 41.4 MB per round for four clients.

By keeping raw data at each junction and exchanging only model parameters, Traffix demonstrates a path towards smarter, privacy-conscious traffic prediction in smart cities.